

SOFTWARE ENGINEERING 2

Prelim Examination

Campus Event Tracker

Project Description

The Campus Event Tracker is a web-based application designed to help campus organization members manage their upcoming events. The application allows users to add new events with titles and dates, view all scheduled events in a list format, and displays appropriate messages when no events are available. This tool does event organization and ensures members stay informed about upcoming campus activities.

Project Scope

Functional Requirements

1. **Event Creation:** Users must be able to add new events by entering an event title and date through input fields and submitting via an "Add" button.
2. **Event Display:** The system will display all added events in a list format showing both the event title and scheduled date for each entry.
3. **Empty State Handling:** When no events exist in the system, the application must display a clear message stating "No events saved" to inform users of the empty state.
4. **Data Persistence:** Events should be stored either in memory or LocalStorage to maintain data during the user session without requiring page refreshes. It also has data input validation in which the user doesn't finish inputting data, it will return an error.

Task Breakdown

Task List

1. **Setup Angular Project Structure** - Initialize the Angular application, configure routing if needed, and set up the basic project structure with necessary folders and files.
2. **Create EventItem Interface** - Define the TypeScript interface for the EventItem model with properties, id (number), title (string), and date (string).
3. **Build Event Tracker Component** - Create the main event-tracker component including the component class, HTML template, and CSS styling file.
4. **Implement Data Binding** - Set up two-way data binding with ngModel for input fields, event binding for the Add button, property binding where needed, and interpolation for displaying event data.

5. **Develop TypeScript Logic** - Implement the `addEvent()` method, initialize the events array, handle event ID generation, and implement data persistence logic (LocalStorage or in-memory).
6. **Design User Interface** - Create the HTML layout with input fields, buttons, event list display, and empty state message. Apply CSS styling or Bootstrap for visual presentation.
7. **Test Application Functionality** - Test adding events, viewing the event list, empty state display, data persistence, and input validation to ensure all features work correctly.
8. **Code Review and Optimization** - Review code quality, optimize performance, ensure proper TypeScript typing, and verify adherence to Angular best practices.

Estimation (Planning Poker)

Task	Story Points	Estimated Time
Setup Angular Project Structure	2	30 minutes
Create EventItem Interface	1	15 minutes
Build Event Tracker Component	3	1 hour
Implement Data Binding	5	1.5 hours
Develop TypeScript Logic	5	1.5 hours
Design User Interface	3	1 hour
Test Application Functionality	3	1 hour
Code Review and Optimization	2	30 minutes

Total Story Points: 24

Total Estimated Time: 7 hours 15 minutes

Planning Poker Scale Used

- 1 point = Very simple task (15 min)
- 2 points = Simple task (30 min)
- 3 points = Moderate task (1 hour)
- 5 points = Complex task (1.5 hours)
- 8 points = Very complex task (2+ hours)

Software Architecture Design

Explanation

Component-Based Architecture in Angular divides the user interface into reusable, self-contained components that each manage their own template, styling, and logic. In the Campus Event Tracker, the event-tracker component structures the application by handling both presentation in its HTML template and business logic in its TypeScript class. Angular's data binding connects the view and component class using an MVVM pattern, enabling two-way communication between the EventItem model and the HTML template.

Justification

Why Component-Based Architecture is Suitable

1. **Simplicity and Maintainability:** For a small application like the Campus Event Tracker with limited functionality, a single-component architecture provides a straightforward structure that is easy to understand, develop, and maintain. All related code (model, logic, and view) is organized in one cohesive unit.
2. **Built-in Angular Support:** Angular is inherently designed around component-based architecture, providing robust tools and features such as decorators, lifecycle hooks, and data binding out of the box. This makes development faster and reduces the need for additional architectural overhead that would be unnecessary for a simple tracker application.
3. **Encapsulation:** The component encapsulates all event-related functionality including data storage, user input handling, and display logic. This encapsulation ensures that the event tracking feature is self-contained and can be easily tested or extended in the future without affecting other parts of the application.
4. **Scalability Potential:** While the current application is simple, the component-based approach allows for future expansion. If the organization needs additional features such as event categories, notifications, or user authentication, new components can be added without restructuring the existing event-tracker component.
5. **Clear Separation of Concerns:** The architecture naturally separates the data model (EventItem interface), the business logic (TypeScript component class), and the presentation (HTML template and CSS), making the codebase organized and following Angular best practices even for a small-scale application.

Architecture Diagram

Model:

- **EventItem Interface**

-id : number

-title : string

-date : string

ViewModel (TypScript):

- Events : EventItem[]
- addEvent()
- Data binding logic
- LocalStorage management

View (HTML Template):

- Input fields (title, date)
- Add Event button
- Event list display
- Empty state message

Data Flow:

View $\longleftrightarrow$ Two-way binding $\longleftrightarrow$ ViewModel

ViewModel $\longleftrightarrow$ Model (EventItem)

ViewModel $\longleftrightarrow$ LocalStorage